DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

1. What is the difference between DELETE and TRUNCATE commands in SQL

Feature	DELETE	TRUNCATE
WHERE clause	✅ Yes	❌ No
Logging	Fully logged (slower)	Minimal logging (faster)
Rollback	✅ Yes	✅ Yes (DB dependent)
Triggers	✅ Fires	❌ Does not fire
Resets Identity	❌ No	✅ Yes
Performance	Slower	Faster

2. How does a Primary Key differ from a Unique Key in SQL?

Feature	Primary Key	Unique Key
NULL allowed	❌ No	✅ Yes (one NULL)
Number per table	One	Multiple
Purpose	Row identification	Enforce uniqueness
Default Index	Clustered	Non-clustered
Used for FK reference	✅ Yes	⚠️ Rarely
Duplicate values	❌ Not allowed	❌ Not allowed

3. What are Views and Common Table Expressions (CTEs), and how do they differ?

Feature	View	CTE
Storage	Stored in DB	Temporary (query scope)
Reusability	✅ Yes	❌ No
Lifetime	Permanent	Single query
Performance	Depends on query	Depends on execution

DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

Feature	View	CTE
Security	Permissions can be applied	No direct permissions
Recursive queries	❌ Not supported	✅ Supported

4. How do DELETE TABLE and TRUNCATE TABLE differ in SQL?

Feature	DELETE TABLE	TRUNCATE TABLE
WHERE clause	✅ Yes	❌ No
Logging	Fully logged	Minimal logging
Triggers	✅ Fires	❌ Does not fire
Resets identity	❌ No	✅ Yes
Performance	Slower	Faster
Rollback	✅ Yes	DB dependent

5. What are the key differences between Stored Procedures and Functions in SQL?

Feature	Stored Procedure	Function
Return value	0 or more	Exactly one
Use in SELECT	❌ Not allowed	✅ Allowed
DML operations	✅ Allowed	❌ Usually not
Call syntax	EXEC proc_name	SELECT function_name()
Transaction control	✅ Supported	❌ Not supported
Use case	Business logic	Calculations / validation

6. How does the HAVING clause differ from the WHERE clause in SQL?

DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

Feature	WHERE	HAVING
Filters	Rows	Groups
Applied	Before GROUP BY	After GROUP BY
Aggregate functions	❌ Not allowed	✅ Allowed
Performance impact	Faster (early filtering)	Slower (post aggregation)
Usage	SELECT, UPDATE, DELETE	SELECT with GROUP BY

7. What is the difference between aggregation functions and analytic (window) functions in SQL

Feature	Aggregation Functions	Analytic Functions
Result	One row per group	One value per row
GROUP BY required	✅ Yes	❌ No
Row count	Reduced	Preserved
OVER() clause	❌ Not used	✅ Required
Use case	Summaries	Ranking, trends, comparisons

8. What are the differences between Star Schema, Snowflake Schema, and Third Normal Form (3NF)?

Feature	Star Schema	Snowflake Schema	3NF
Structure	Fact + denormalized dimensions	Fact + normalized dimensions	Fully normalized tables
Complexity	Simple	Medium	High
Query performance	Fast	Slightly slower	Slow for analytics
Joins	Fewer	More	Many

DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

Feature	Star Schema	Snowflake Schema	3NF
Data redundancy	Higher	Lower	Minimal
Primary use	OLAP / DW	OLAP / DW	OLTP systems

9. Can you explain the ACID properties of transactions in SQL with an example?

- **Atomicity:** If either update fails, both are rolled back.
- **Consistency:** Total balance across accounts remains correct.
- **Isolation:** Other transactions don't see partial updates.
- **Durability:** Once committed, the transfer is permanent.

10. How do you create and use clustered and non-clustered indexes in SQL?

Clustered Index

- **Physically sorts and stores table data** based on the index key.
- A table can have **only one clustered index**.
- Typically created on **primary key columns**.
- Very efficient for **range queries and sorting**

Non-Clustered Index

- Creates a **separate index structure** that points to the actual data rows.
- A table can have **multiple non-clustered indexes**.
- Does **not affect the physical order** of the table.
- Useful for **frequently searched columns**.

11. What is the difference between UNION and UNION ALL in SQL?

UNION

- Combines result sets and **removes duplicate rows**.
- Performs an implicit **DISTINCT** operation.

- Requires additional sorting/comparison, so it is **slower**.
- Best used when **unique results** are required.

UNION ALL

- Combines result sets and keeps all rows, including duplicates.
- No deduplication, so it is faster.
- Preferred when duplicates are acceptable or meaningful.

12. What are common concurrency issues in SQL databases, and how can they be handled?

Common Concurrency Issues

1️⃣ **Dirty Reads**

- A transaction reads **uncommitted data** from another transaction.
- If the other transaction rolls back, the read data becomes invalid.

2️⃣ **Lost Updates**

- Two transactions update the same data.
- One update **overwrites** the other without awareness.

3️⃣ **Non-Repeatable Reads**

- A transaction reads the same row twice and gets **different values** due to another committed update.

4️⃣ **Deadlocks**

- Two or more transactions **wait on each other's locks**, causing none to proceed.

13. What are constraints in SQL?

Constraints enforce data integrity rules. Common types include:

- **NOT NULL** – prevents NULL values
- **UNIQUE** – ensures uniqueness
- **PRIMARY KEY** – unique + not null

- **FOREIGN KEY** – enforces referential integrity
- **CHECK** – enforces conditions
- **DEFAULT** – assigns default values

14. **Difference between DELETE, DROP, and TRUNCATE?**

- **DELETE**: Removes rows (can use WHERE)
- **TRUNCATE**: Removes all rows quickly
- **DROP**: Removes the table structure itself

15. **Difference between EXISTS and IN?**

- **EXISTS** checks for existence and stops early
- **IN** compares values
  EXISTS performs better for large datasets.

16. **What are isolation levels?**
Isolation levels control concurrency:

- READ UNCOMMITTED
- READ COMMITTED
- REPEATABLE READ
- SERIALIZABLE

17. **What are different types of joins?**

- **INNER** – matching rows
- **LEFT / RIGHT** – all rows from one side
- **FULL** – all rows from both sides
- **CROSS** – Cartesian product
- **SELF** – table joined to itself

DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

SQL CODING QUESTIONS:

1. How can you find the second-highest salary in an employee table using SQL?"

Assume table: employees(emp_id, emp_name, salary)

✅ **Method 1: Using DISTINCT with LIMIT / OFFSET**

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

✅ **Method 2: Using MAX() with a subquery**

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

✅ **Method 3: Using DENSE_RANK()**

```
SELECT salary
FROM (
  SELECT salary,
      DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) ranked
```

```
WHERE rnk = 2;
```

2. How can you retrieve the top 3 highest-paid employees in each department using SQL

```
SELECT *
FROM (
  SELECT *,
      DENSE_RANK() OVER (
        PARTITION BY department
        ORDER BY salary DESC
      ) AS rnk
  FROM employees
) sub
WHERE rnk <= 3;
```

3. How can you retrieve the Nth highest salary from an employees table using SQL?

```
SELECT salary
FROM (
  SELECT salary,
      DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) ranked_salaries
WHERE rnk = N;
```

4. How do you identify and count duplicate records in a table, for example duplicate employee names?

```
SELECT emp_name, COUNT(*) AS duplicate_count
FROM employees
```

```
GROUP BY emp_name
HAVING COUNT(*) > 1;
```

USING LAG and RANK

```
SELECT emp_id,
    emp_name,
    salary,
    LAG(salary) OVER (PARTITION BY department ORDER BY salary) AS prev_salary,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS salary_rank
FROM employees;
```

5. How can you retrieve records that are present in one table but missing in another table?

✅ **Method 1: Using LEFT JOIN (Most Common)**

```
SELECT a.*
FROM table_a a
LEFT JOIN table_b b
  ON a.id = b.id
WHERE b.id IS NULL;
```

✅ **Method 2: Using NOT IN**

```
SELECT *
FROM table_a
WHERE id NOT IN (SELECT id FROM table_b);
```

✅ **Method 3: Using NOT EXISTS**

```
SELECT *
FROM table_a a
```

```
WHERE NOT EXISTS (
  SELECT 1
  FROM table_b b
  WHERE a.id = b.id
);
```

6. How can you retrieve employees who earn more than the company's average salary?

```
SELECT *
FROM employees
WHERE salary > (
  SELECT AVG(salary)
  FROM employees
);
```

7. find employees whose salary is higher than their manager's salary?

```
SELECT e.employee_id,
    e.name AS employee_name,
    e.salary AS employee_salary
FROM employees e
JOIN employees m
   ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

8. Remove duplicate records from a table while keeping only one unique row?

```
WITH cte AS (
  SELECT *,
      ROW_NUMBER() OVER (
```

```
        PARTITION BY column_name
        ORDER BY column_name
      ) AS row_num
  FROM table_name
)
DELETE
FROM cte
WHERE row_num > 1;
```

9. Identify the top three highest-paid employees in each department?

```
SELECT *
FROM (
  SELECT *,
      DENSE_RANK() OVER (
        PARTITION BY department
        ORDER BY salary DESC
      ) AS rnk
  FROM employees
) ranked
WHERE rnk <= 3;
```

10. How can you retrieve records that exist in both tables

```
SELECT *
FROM table1
INTERSECT
SELECT *
FROM table2;
```

11. Transform row-based data into column-based data using a PIVOT operation?

```
SELECT *
FROM (
  SELECT department, month, salary
  FROM salaries
) src
PIVOT (
  SUM(salary)
  FOR month IN ([Jan], [Feb], [Mar])
) AS pvt;
```

12. How can you fetch the most recent 10 records from a table?

```
SELECT *
FROM employees
ORDER BY employee_id DESC
LIMIT 10;
```

13. Retrieve rows that exist in one table but do not exist in another table

```
SELECT *
FROM table1
WHERE id NOT IN (
  SELECT id
  FROM table2
);
```

14. How can you retrieve the five highest-paid employees from a table?

```
SELECT *
FROM employees
```

```
ORDER BY salary DESC
LIMIT 5;
```

15. Calculate a running (cumulative) total of salaries within each department?

```
SELECT employee_id,
    department,
    salary,
    SUM(salary) OVER (
      PARTITION BY department
      ORDER BY employee_id
    ) AS cumulative_salary
FROM employees;
```

16. Retrieve employees who were hired in a specific year (for example, 2025)

```
SELECT *
FROM employees
WHERE YEAR(join_date) = 2025;
```

17. Find customers who made purchases in the previous year but have not made any purchases in the current year?

```
SELECT ly.customer_id
FROM purchases ly
LEFT JOIN purchases cy
   ON ly.customer_id = cy.customer_id
  AND cy.year = 2023
WHERE ly.year = 2022
 AND cy.customer_id IS NULL;
```

18. Retrieve employees who have joined within the last six months?

```
SELECT *
FROM employees
WHERE joining_date >= DATEADD(MONTH, -6, GETDATE());
```

19. Identify the department that has the maximum number of employees

```
SELECT department_id,
    COUNT(*) AS employee_count
FROM employees
GROUP BY department_id
ORDER BY employee_count DESC
LIMIT 1;
```

20. Identify customers who have purchased every product from a specific list of products?

```
SELECT customer_id
FROM purchases
WHERE product_id IN (1, 2, 3)   -- target product list
GROUP BY customer_id
HAVING COUNT(DISTINCT product_id) = 3;
```

21. Find the highest salary in each department?

```
SELECT department,
    MAX(salary) AS max_salary
FROM employees
GROUP BY department;
```

22. check number of employees working in each department?

```
SELECT department_id,
    COUNT(*) AS employee_count
FROM employees
GROUP BY department_id;
```

23. consecutive absence days for employees using attendance data?

```
SELECT employee_id,
    date,
    status,
    LAG(status) OVER (
      PARTITION BY employee_id
      ORDER BY date
    ) AS prev_status
FROM attendance
WHERE status = 'absent';
```

24. Join two related tables and retrieve only selected columns from each

```
SELECT e.employee_id,
    e.name AS employee_name,
    d.department_name
FROM employees e
JOIN departments d
   ON e.department_id = d.department_id;
```

25. Retrieve employees who earn the highest salary within their respective departments

```
SELECT e.department_id,
    e.employee_id,
```

```
    e.salary
FROM employees e
WHERE e.salary = (
  SELECT MAX(salary)
  FROM employees
  WHERE department_id = e.department_id
);
```

26. Find orders that have been dispatched but have not yet been invoiced?

```
SELECT o.order_id
FROM orders o
LEFT JOIN billing b
   ON o.order_id = b.order_id
WHERE o.status = 'shipped'
 AND b.order_id IS NULL;
```

27. Identify missing numbers in a continuous numeric sequence stored in a table

```
SELECT n.number + 1 AS missing_number
FROM numbers n
WHERE n.number + 1 NOT IN (
  SELECT number FROM numbers
);
```

28. Retrieve the youngest employee in an organization?

```
SELECT *
FROM employees
ORDER BY birth_date DESC
LIMIT 1;
```

29. Performance-based rankings to employees within each department

```
SELECT employee_id,
    department_id,
    performance_score,
    RANK() OVER (
      PARTITION BY department_id
      ORDER BY performance_score DESC
    ) AS dept_rank
FROM employees;
```

30. calculate the average salary for each department

```
SELECT department,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

31. Retrieve the earliest and most recent records from a table

```
(SELECT *
 FROM employees
 ORDER BY employee_id ASC
 LIMIT 1)
UNION ALL
(SELECT *
 FROM employees
 ORDER BY employee_id DESC
 LIMIT 1);
```

32. Identify the earliest purchase made by each customer

```
SELECT customer_id, purchase_date, amount
FROM (
  SELECT customer_id,
      purchase_date,
      amount,
      ROW_NUMBER() OVER (
        PARTITION BY customer_id
        ORDER BY purchase_date ASC
      ) AS row_num
  FROM purchases
) first_purchases
WHERE row_num = 1;
```

33. Calculate the median salary of employees using SQL

```
SELECT DISTINCT
    PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY salary)
    OVER () AS median_salary
FROM employees;
```

34. identify the department that has the lowest average salary

```
SELECT department_id,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
ORDER BY avg_salary ASC
LIMIT 1;
```

35. Calculate the average sales for each month of every year

```
SELECT YEAR(sale_date) AS sale_year,
    MONTH(sale_date) AS sale_month,
    AVG(sales) AS avg_sales
FROM sales
GROUP BY YEAR(sale_date), MONTH(sale_date)
ORDER BY sale_year, sale_month;
```

36. find all employees who do not have a manager assigned?

```
SELECT *
FROM employees
WHERE manager_id IS NULL;
```

37. Retrieve the second-highest value from a column in a table

```
SELECT MAX(column_name) AS second_highest
FROM table_name
WHERE column_name < (SELECT MAX(column_name) FROM table_name);
```

38. Calculate a running total of sales over time using SQL

```
SELECT sale_date,
    sales,
    SUM(sales) OVER (ORDER BY sale_date) AS running_total
FROM sales;
```

39. Create a new table from the results of a query in SQL

```
SELECT department,
    AVG(salary) AS avg_salary
```

```
INTO DepartmentSalary
FROM employees
GROUP BY department;
```

40. Retrieve all employees who do not manage any subordinates

```
SELECT *
FROM employees
WHERE employee_id NOT IN (
  SELECT DISTINCT manager_id
  FROM employees
  WHERE manager_id IS NOT NULL
);
```

41. Find products that appear in multiple orders with the same price

```
SELECT product_id, price, COUNT(*) AS order_count
FROM orders
GROUP BY product_id, price
HAVING COUNT(*) > 1;
```

42. check if a table contains any rows or is empty using SQL

```
SELECT CASE
      WHEN EXISTS (SELECT 1 FROM table_name)
      THEN 'Not Empty'
      ELSE 'Empty'
    END AS table_status;
```

43. Retrieve all orders placed within the last 7 days?

```
SELECT *
```

```
FROM orders
WHERE order_date >= DATEADD(DAY, -7, GETDATE());
```

44. Total number of employees for each job title

```
SELECT job_title, COUNT(*) AS total_employees
FROM employees
GROUP BY job_title;
```

45. Retrieve the highest-value order for each customer

```
SELECT customer_id, order_id, amount
FROM (
  SELECT customer_id, order_id, amount,
      ROW_NUMBER() OVER (PARTITION BY customer_id
              ORDER BY amount DESC) AS row_num
  FROM orders
) AS ranked_orders
WHERE row_num = 1;
```

46. Find employees who are assigned to both departments 101 and 102?

```
SELECT employee_id
FROM employees
WHERE department_id IN (101, 102)
GROUP BY employee_id
HAVING COUNT(DISTINCT department_id) = 2;
```

47. Find the best-selling product for each month?

```
WITH monthly_sales AS (
  SELECT product_id,
```

```
    MONTH(sale_date) AS month,
    SUM(quantity) AS total_quantity,
    ROW_NUMBER() OVER (PARTITION BY MONTH(sale_date)
            ORDER BY SUM(quantity) DESC) AS rank
  FROM sales
  GROUP BY product_id, MONTH(sale_date)
)
SELECT product_id, month, total_quantity
FROM monthly_sales
WHERE rank = 1;
```

48. Retrieve all employees who share the same salary with at least one other employee

```
SELECT *
FROM employees
WHERE salary IN (
  SELECT salary
  FROM employees
  GROUP BY salary
  HAVING COUNT(*) > 1
);
```

49. Calculate customer cohort retention rate using SQL

```
SELECT cohort_month,
   COUNT(DISTINCT retained.customer_id) * 1.0 / COUNT(DISTINCT
initial.customer_id) AS retention_rate
FROM (
  SELECT customer_id, DATE_TRUNC('month', signup_date) AS cohort_month
```

```
   FROM customers
) AS initial
LEFT JOIN (
   SELECT customer_id, DATE_TRUNC('month', purchase_date) AS purchase_month
   FROM customers
) AS retained
ON initial.customer_id = retained.customer_id
  AND retained.purchase_month >= initial.cohort_month
GROUP BY cohort_month
ORDER BY cohort_month;
```

50. Update employee salaries differently based on their department in a single query

```
UPDATE employees
SET salary = CASE
          WHEN department_id = 101 THEN salary * 1.10
          WHEN department_id = 102 THEN salary * 1.05
          ELSE salary
        END;
```